Snapdeal Automation Framework Project

As

Veeva Systems Assignment - Web Test Automation

Submitted By
Valluri Gowthami Rushitha
from
AutoTest Duo Team

Duration
10-03-2026 to 10-04-2026

Teammate
Yadavalli Lakshmi Vyshnavi

Veeva Systems

Veeva Systems is a leading provider of cloud-based software solutions designed specifically for the global life sciences industry. Founded in 2007 and headquartered in Pleasanton, California, Veeva focuses on helping pharmaceutical, biotechnology, and medical device organizations streamline their operations, maintain regulatory compliance, and accelerate the delivery of therapies to patients.

Veeva's comprehensive product suite includes solutions for customer relationship management (CRM), clinical data management, regulatory affairs, quality management, and commercial operations. Built on a strong foundation of innovation, data integrity, and compliance with global standards such as GxP and FDA regulations, Veeva enables organizations to enhance efficiency while ensuring the highest levels of quality and reliability. With a customer-centric approach and continuous technological advancements, Veeva has established itself as a trusted partner for life sciences companies worldwide.

Introduction

In today's fast-paced software development environment, ensuring the quality and reliability of web applications is essential. Automation testing plays a crucial role in achieving this by enabling faster execution of test cases, improving accuracy, and supporting continuous integration and delivery. A well-designed automation framework enhances reusability, scalability, and maintainability, making it an indispensable component of modern software quality assurance.

This project focuses on the design and implementation of a Hybrid Web Test Automation Framework using Java, Selenium WebDriver, Cucumber (BDD), Maven, and Git. The automated test scenarios simulate real-world user interactions on an e-commerce platform, such as searching for products and adding items to the cart. The framework supports cross-browser execution, external test data management, comprehensive reporting, and logging mechanisms, thereby reflecting industry best practices.

Overall, this project demonstrates the ability to build a scalable and maintainable automation solution that aligns with real-world software testing standards and Veeva's emphasis on quality, compliance, and reliability.

Objective of the Assignment

This project aims to design and implement a robust, scalable, and maintainable web automation testing framework for the Snapdeal e-commerce platform. The framework adheres to industry best practices by leveraging a hybrid approach that combines the Page Object Model (POM), Behavior-Driven Development (BDD) using Cucumber, and Data-Driven Testing. It demonstrates proficiency in modern automation technologies, cross-browser compatibility, Maven-based execution, comprehensive reporting and logging, and the application of core object-oriented programming principles. The project also emphasizes reusability, configurability, and alignment with real-world software testing environments.

Table of Contents

S.No	Contents	Page No
1	**Project Overview**	**1**
1.1	Introduction	1
1.2	Objective of the Assignment	1
1.3	Tools & Technologies	2
1.4	Project Setup & Configuration	3
1.5	Project Structure	3
2	**Contribution**	**4**
2.1	Team Contribution	4
2.2	My Contribution	5
3	**Architecture & Design**	**6**
3.1	Framework Architecture	6
3.2	Execution Flow	7
4	**Framework Engineering Depth**	**12**
4.1	Automation Framework Concepts	12
4.2	Framework Components	14
4.3	Maven Execution & Build Life cycle	15
5	**Testing Strategy**	**16**
5.1	Test Strategy & Approach	16
5.2	Test Scenarios Implementation	17
5.3	Test Coverage & Priortization Strategy	19
6	**Data & Metrics**	**20**
6.1	Data Driven Testing	20
6.2	Test Data Management Strategy	22
6.3	Test Execution Metrics & Analysis	23
7	**Advanced Capabilities**	**25**
7.1	Cross browser support	25
7.2	Wait handling & Synchronization strategy	26
7.3	Error handling strategy	27

8	**Reporting & Monitoring**	**28**
8.1	Reporting & Logging	28
8.2	Execution Reports & Insights	33
9	**Configuration & Maintainability**	**34**
9.1	Configuration Management	34
9.2	Framework Scalability & Reusability	35
10	**CI/CD Integration**	**36**
10.1	Github Actions	36
11	**Real World Considerations**	**38**
11.1	Risk Analysis	38
11.2	Challenges faced & Solutions	39
11.3	Limitations & Future Enhancements	42
12	**Business Justification**	**43**
12.1	Design Decisions (Why snapdeal over Walmart)	43
13	**Outcomes**	**45**
13.1	Learning outcomes	45
13.2	Execution proof	46
13.3	Conclusion	48

List of Figures

Figure No	Title	Page No
Fig 1.5.1	Project Structure	4
Fig 3.1.1	Framework Architecture	7
Fig 3.2.1	Execution Flow (Scenario 1)	8
Fig 3.2.2	Execution Flow (Scenario 2)	9
Fig 4.2.1	Interaction of framework components	14
Fig 4.3.1	Maven Build Life Cycle	15
Fig 6.1.1	Data Driven Testing Flow	21
Fig 6.3.1	Metrics Analysis	24
Fig 8.1.1	Extent Report	29
Fig 8.1.2	Cucumber Report	30
Fig 8.1.3	Surefire Report	31
Fig 8.1.4	Logging with Log4j2.xml	31
Fig 8.1.5	Reporting & Logging Flow	32
Fig 10.1.1	CI/CD pipeline Flow	37
Fig 13.2.1	Execution Proof	47

List of Tables

Table No	Title	Page No
Tab 1.3.1	Tools & Technologies	2
Tab 4.1.1	OOPS Principles	13
Tab 4.2.1	Framework Components	14
Tab 4.3.1	Maven Build Lifecycle phases	15
Tab 4.3.2	Maven Execution Commands	16
Tab 5.3.1	Priority levels	19
Tab 5.3.2	Test coverage matrix	19
Tab 6.1.1	Components implemented in our framework	20
Tab 6.1.2	TestData for Scenario 2	21
Tab 6.2.1	Data Organization	22
Tab 6.3.1	Key Metrics for Scenario 2	23
Tab 6.3.2	Overall Metrics Table (2 Scenarios)	24
Tab 7.2.1	Synchronization techniques used in Scenario 2	17
Tab 7.3.1	Exceptions Handled	27
Tab 8.1.1	Reporting tools used	29
Tab 8.1.2	Logging levels	32
Tab 8.2.1	Scenario wise Execution summary	33
Tab 9.1.1	Key Configuration elements	34
Tab 9.2.1	Reusable utility classes	36
Tab 10.1.1	Explanation of workflow steps	38
Tab 10.1.2	Artifact management	38
Tab 11.1.1	Risks & Mitigation Strategies	39
Tab 11.1.2	Risk Severity Matrix	39

Section 1 : Project Overview

1.1 Introduction :

This project presents the design and implementation of a scalable and maintainable Selenium-based automation framework for the Snapdeal e-commerce platform. The framework is developed using industry-standard tools and best practices, including the Page Object Model (POM) design pattern, data-driven testing, and cross-browser compatibility. The primary objective is to automate critical user journeys such as product search, product selection, cart management, and checkout processes.

This project demonstrates practical expertise in test automation and reflects the quality engineering standards expected in enterprise environments.

1.2 Objective of the Assignment :

The primary objective of this assignment is to design and implement a robust, scalable, and maintainable web automation testing framework that adheres to industry best practices. The framework aims to simulate real-world testing environments and demonstrate proficiency in automation technologies and software engineering principles.

Key Objectives :

1. **Develop a Hybrid Automation Framework** : Implement a combination of Page Object Model (POM), Behavior-Driven Development (BDD) using Cucumber, and Data-Driven Testing to ensure modularity and reusability.

2. **Utilize Modern Automation Tools and Technologies** : Employ Java for programming, Selenium WebDriver for browser automation, Maven for build and dependency management, and Git for version control.
3. **Ensure Cross-Browser Compatibility** : Enable test execution across multiple browsers such as Google Chrome and Microsoft Edge, with browser selection controlled through configuration files.

4. **Implement Robust Reporting and Logging :** Generate detailed execution reports and logs using tools such as Extent Reports and Log4j, along with automatic screenshot capture on test failures.
5. **Demonstrate OOPs Principles** : Apply core object-oriented programming concepts including Encapsulation, Abstraction, Inheritance, Polymorphism, and Interfaces to enhance maintainability and scalability.
6. **Support Maven-Based Execution** : Ensure seamless execution of the framework using Maven commands such as mvn clean test, validating the build lifecycle.
7. **Incorporate External Test Data** : Facilitate Data-Driven Testing by integrating external data sources such as Excel or JSON files for parameterization of test scenarios.
8. **Adopt Industry Best Practices** : Maintain a clean project structure, reusable components, meaningful assertions, and proper error-handling mechanisms.

1.3 Tools & Technologies :

The automation framework leverages a combination of industry-standard tools and technologies to ensure robustness, scalability, and maintainability.

Category	Tool/Technology	Purpose
Programming Language	Java	Core language for automation scripts
Automation Tool	Selenium WebDriver	Browser automation
Test Framework	TestNG	Test execution and assertions
Build Tool	Maven	Dependency management and build lifecycle
Design Pattern	Page Object Model (POM)	Enhances maintainability and reusability
Data Handling	Apache POI	Reading data from Excel files
Reporting	Extent Reports	Rich and interactive test reports
Logging	Log4j	Logging execution details
Version Control	Git & GitHub	Source code management
CI Tool	GitHub Actions	Automated test execution
Browser Drivers	ChromeDriver, EdgeDriver	Cross-browser testing

Tab 1.3.1

1.4 Project Setup & Configuration :

The automation framework is built using Maven for dependency management and follows a structured configuration approach to ensure flexibility and ease of maintenance.

Setup Steps :

1.Install Prerequisites

1. Java Development Kit (JDK 8 or above)
2. Maven
3. IDE (Eclipse or IntelliJ)
4. Web browsers (Chrome, Edge)

2.Clone the Repository

```
git clone https://github.com/Rushitha-Valluri64/Veeva-Project.git
cd SNAPDEAL
```

3.**Install Dependencies**

```
mvn clean install
```

4.Configuration Files

1. config.properties for environment settings.
2. testng.xml for test suite configuration.
3. pom.xml for dependency management.

5.Running the Tests

1. From IDE: Execute through runner class
2. From Command Line: mvn clean test

6.GitHub Actions Integration

1. Automated execution of tests on every push or pull request.
2. Generation of reports as build artifacts.

1.5 Project Structure :

The project follows the Page Object Model (POM) design pattern, ensuring separation of concerns and improved maintainability. The directory structure is organized to promote reusability and scalability.

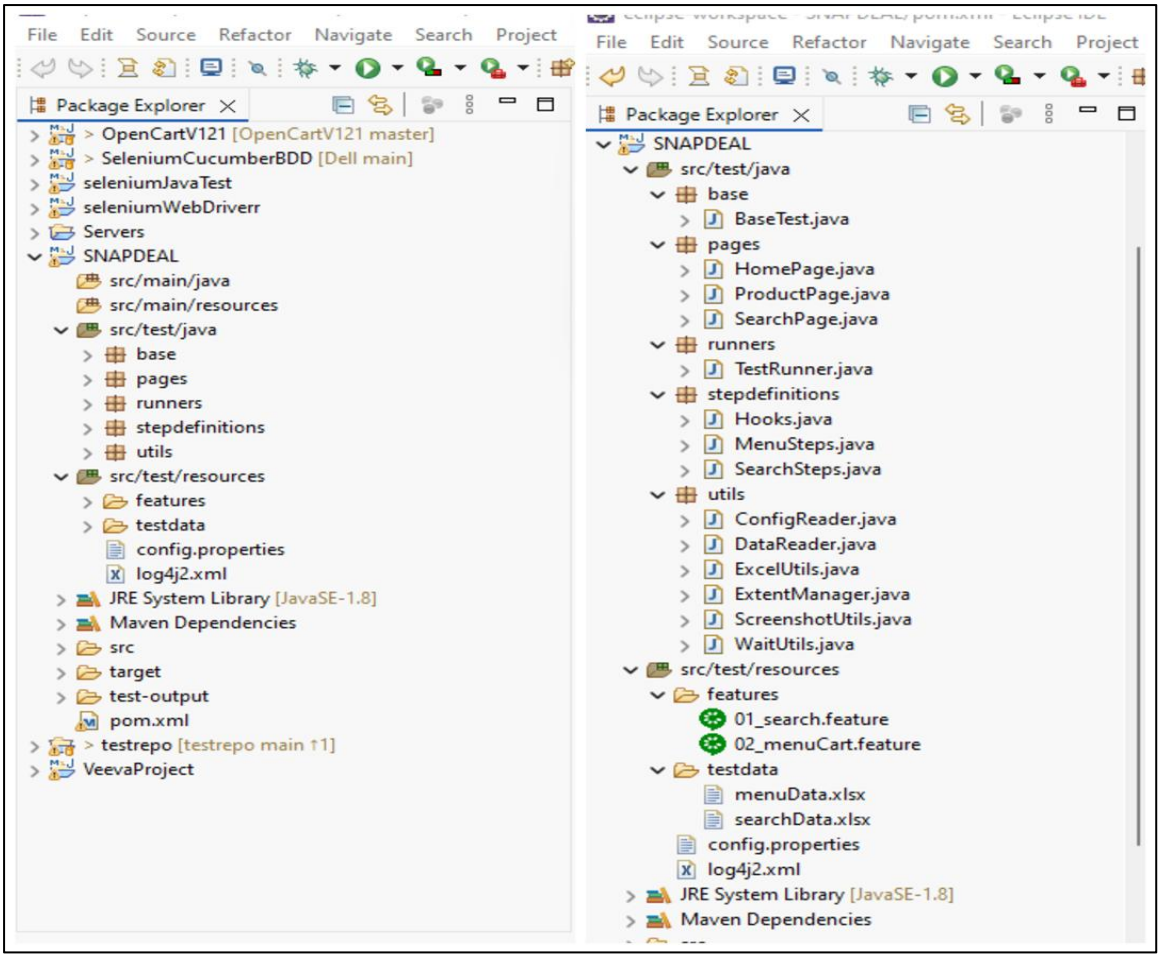


Fig 1.5.1

Section 2 : Contribution

1.1 Team Contribution :

The Snapdeal web automation framework was developed as a collaborative effort by a team of two members, following the guidelines of the Veeva Web Test Automation assignment. The responsibilities were divided to ensure efficient implementation and comprehensive coverage of the required test scenarios.

- Yadavalli Lakshmi Vyshnavi (Teammate) was responsible for the implementation of Scenario 1 – Product Search Functionality, which involved automating the search of a product on the Snapdeal website and validating the relevance of the displayed results.

- Valluri Gowthami Rushitha (me) was responsible for the implementation of Scenario 2 – Add Product to Cart, which involved automating the end-to-end workflow from product selection to cart validation.

In addition to scenario-specific responsibilities, both team members collaboratively contributed to discussions on framework design, adherence to best practices, and successful integration of the scenarios into a unified hybrid automation framework.

1.2 My Contribution :

I was primarily responsible for the design and implementation of Scenario 2 – Add Product to Cart for the Snapdeal e-commerce platform. My work focused on automating the complete user journey from product selection to cart validation while ensuring the framework's scalability, maintainability, and adherence to industry best practices.

Scenario 2 Implementation :

- Analyzed the functional workflow of the Snapdeal application to design effective automation steps.
- Developed Cucumber feature files using Gherkin syntax to describe the behavior of the add-to-cart functionality.
- Implemented corresponding step definitions to map user actions to Selenium WebDriver operations.
- Created Page Object Model (POM) classes for key pages.
- Automated navigation through product categories and selection of a product to simulate real user behavior.
- Implemented validations to ensure:
    - Successful navigation to the Product Details Page.
    - Correct addition of the product to the shopping cart.
    - Accurate cart count displayed in the header.
    - Verification of subtotal and estimated total values in the cart.

Framework Development and Enhancements :

- Contributed to the development of the Hybrid Automation Framework integrating Page Object Model (POM), Behavior-Driven Development (BDD) using Cucumber, and Data-Driven Testing.

- Developed reusable utility classes for WebDriver initialization, synchronization using explicit waits, screenshot capture, and configuration management.
- Implemented cross-browser support for Google Chrome and Microsoft Edge, with browser selection controlled through the config.properties file.
- Integrated Extent Reports to generate detailed and visually rich execution reports.
- Configured Log4j for effective logging of test execution steps and debugging.
- Enabled Maven-based execution of the test suite using the mvn clean test command.
- Utilized Git for version control, ensuring meaningful commits and proper repository management.
-

Section 3 : Architecture & Design

3.1 Framework Architecture :

The Snapdeal automation project is built using a Hybrid Automation Framework that integrates the Page Object Model (POM), Behavior-Driven Development (BDD) using Cucumber, and Data-Driven Testing. This architecture was collaboratively designed by both team members to ensure scalability, maintainability, and reusability of the automation suite.

The framework supports multiple test scenarios, including Scenario 1 – Product Search, implemented by Yadavalli Lakshmi Vyshnavi, and Scenario 2 – Add Product to Cart, implemented by Valluri Gowthami Rushitha. While the architectural foundation is shared, each scenario leverages this structure to automate specific business workflows.

Architectural Layers :

1. **Feature Layer :** Contains Cucumber feature files written in Gherkin syntax for both scenarios.
2. **Step Definition Layer :** Maps Gherkin steps to Java methods.
3. **Page Object Layer :** Encapsulates web elements and actions for Snapdeal pages.
4. **Base/Test Layer :** Manages WebDriver initialization and teardown.
5. **Utility Layer :** Provides reusable components like waits, screenshots, configuration readers, and data handlers.
6. **Data Layer :** Supports external test data (Excel).
7. **Reporting & Logging Layer :** Integrates Extent Reports and Log4j.

8. **Build & CI Layer :** Utilizes Maven and GitHub Actions for automated execution.

The below diagram illustrates the layered architecture of the Snapdeal automation framework, integrating Selenium WebDriver, Cucumber BDD, TestNG, and the Page Object Model. It supports both Scenario 1 (Product Search) and Scenario 2 (Menu Navigation and Cart Validation) with shared utilities, data-driven capabilities, and comprehensive reporting.

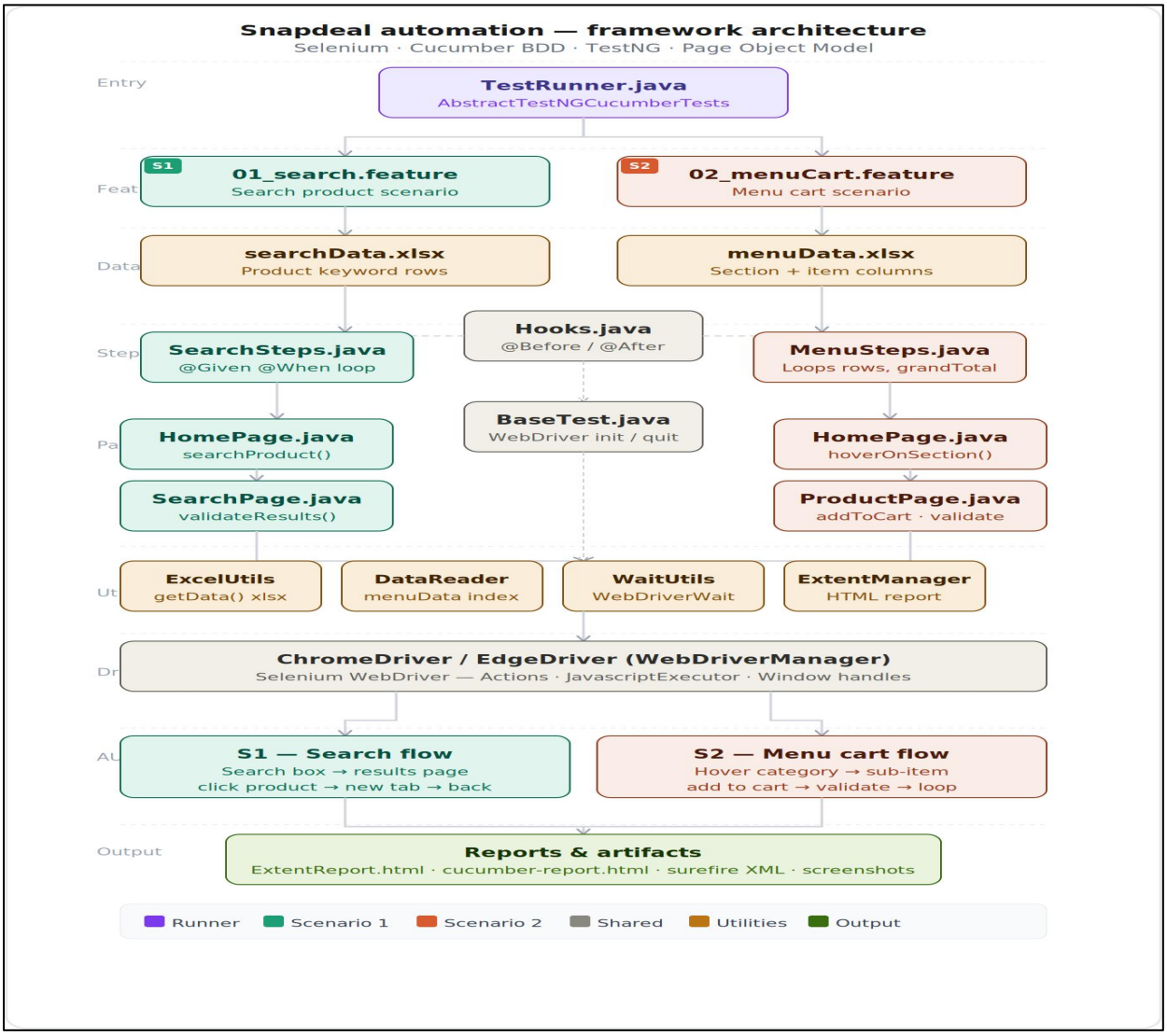


Fig 3.1.1

The Snapdeal automation framework architecture demonstrates a robust and industry-aligned design that effectively supports scalable and maintainable test automation. By integrating modern tools and best practices, the framework ensures reliable validation of critical e-commerce workflows while clearly distinguishing the contributions of each team member.

3.2 Execution Flow :

Scenario 1 – Product Search (Teammate: Yadavalli Lakshmi Vyshnavi)

This scenario validates the search functionality of the Snapdeal application. It was implemented by Yadavalli Lakshmi Vyshnavi and ensures that users can search for a product and view relevant results.

Steps:

1. Launch the Snapdeal homepage.
2. Enter a product keyword in the search bar.
3. Select a suggestion or initiate the search.
4. Verify that the search results page is displayed.
5. Validate that the displayed products are relevant to the search keyword.

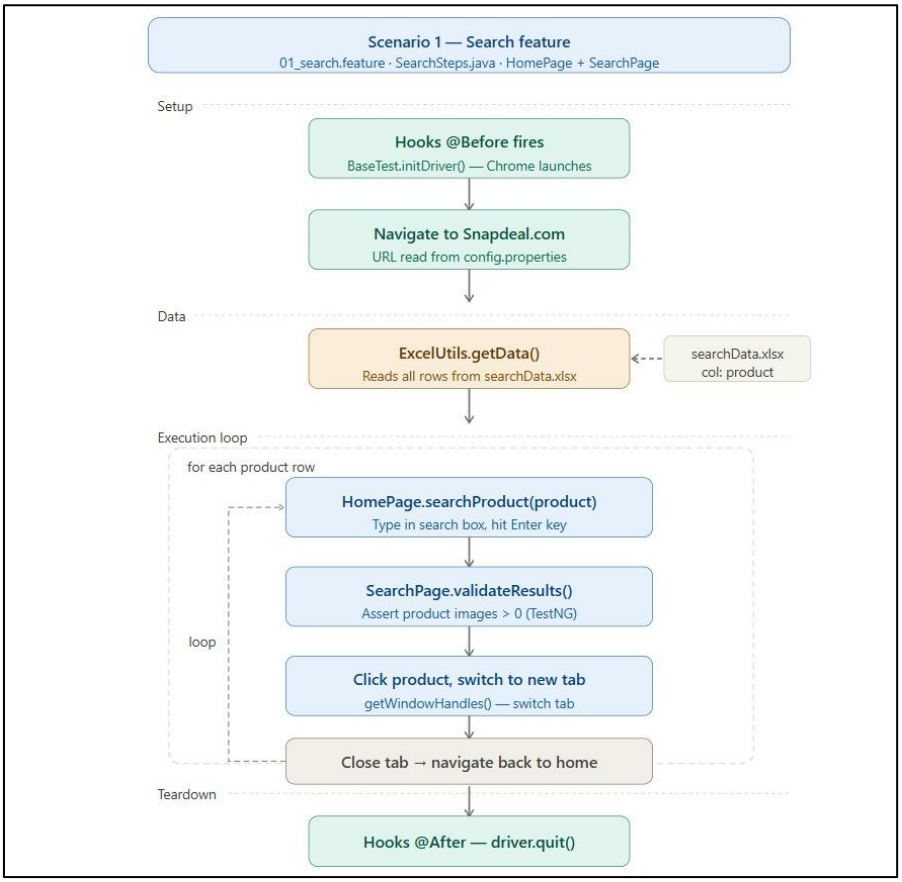


Fig 3.2.1

This diagram depicts the execution lifecycle of the product search functionality on the Snapdeal platform. It includes WebDriver initialization through Cucumber hooks, navigation to the application, retrieval of search data from an Excel file, execution of search operations using Page Object methods, validation of search results, and proper teardown of the browser session.

Scenario 2 – Add Product to Cart (My Work: Valluri Gowthami Rushitha)

This scenario, implemented by Valluri Gowthami Rushitha (me) , validates the end-to-end workflow of selecting a product and adding it to the shopping cart, including verification of cart details.

Steps :

1. Launch the Snapdeal homepage.
2. Navigate through product categories by hovering the mouse.
3. Select a product from the listing page.
4. Verify navigation to the Product Details Page.
5. Click on the Add to Cart button.
6. Validate that the cart count in the header is updated.
7. Verify the subtotal and estimated total displayed in the cart.
8. Ensure that the correct product is added.

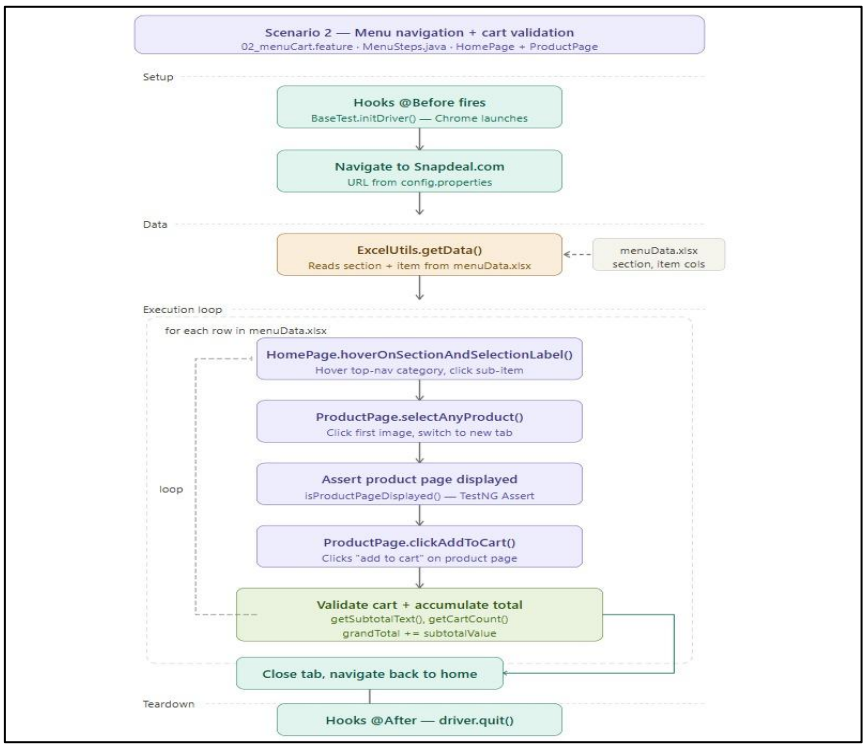


Fig 3.2.2

This diagram illustrates the end-to-end automation flow for adding a product to the cart on the Snapdeal platform. The process includes WebDriver initialization, navigation to the application, data-driven menu navigation, product selection, validation of the product details page, addition of the product to the cart, verification of cart count and totals, and graceful teardown of the browser session.

Detailed Execution Flow (Scenario 2) :

Step 1: Test Initialization

- The Cucumber @Before hook triggers the setup process.
- The BaseTest.initDriver() method initializes the WebDriver based on the browser specified in config.properties (e.g., Chrome or Edge).
- The browser window is maximized, and implicit/explicit waits are configured.

Step 2: Navigation to Snapdeal

- The application URL is read from the config.properties file.
- The browser navigates to the Snapdeal homepage.
- A validation ensures that the homepage is successfully loaded.

Step 3: Data-Driven Menu Navigation

- Test data is retrieved from menuData.xlsx using ExcelUtils.getData().
- Each row in the Excel file contains:
    - Section (e.g., “Men”, “Electronics”)
    - Item/Subcategory (e.g., “Shirts”, “Mobiles”)
- The scenario iterates through each dataset, enabling parameterized execution.

Step 4: Hover and Subcategory Selection

- The method HomePage.hoverOnSectionAndSelectionLabel() performs:
    - Hovering over the top navigation menu using the Actions class.
    - Clicking the specified subcategory.
- This simulates real user interaction with dynamic menus.

Step 5: Product Selection

- The ProductPage.selectAnyProduct() method:
    - Selects the first available product from the listing page.
    - Handles navigation to a new browser tab using getWindowHandles().

- Switches the WebDriver context to the product details page.

Step 6: Product Details Page Validation

- The method isProductPageDisplayed() verifies that:
  - The product title and key elements are visible.
  - The user is on the correct Product Details Page (PDP).
- TestNG assertions ensure the correctness of navigation.

Step 7: Add Product to Cart

- The ProductPage.clickAddToCart() method clicks the "Add to Cart" button.
- Explicit waits ensure that the button is clickable and the action is successfully completed.
- A confirmation message or cart overlay is validated.

Step 8: Cart Validation

The scenario performs multiple validations:

- Cart Count Validation : Confirms that the cart icon reflects the correct number of items.
- Subtotal Verification : Extracts the subtotal value using getSubtotalText().
- Grand Total Calculation : Accumulates subtotal values to compute the estimated total.
- Product Verification : Ensures that the correct product is listed in the cart.

Step 9: Iterative Execution

- The process repeats for each row in the Excel dataset, enabling data-driven testing and broader coverage.

Step 10: Cleanup and Teardown

- After scenario execution, the Cucumber @After hook is triggered.
- The browser is closed using driver.quit().
- Execution results are captured in Extent Reports, and logs are recorded via Log4j.

Section 4 : Framework Engineering Depth

4.1 Automation Framework Concepts :

The Snapdeal automation project is built using a Hybrid Automation Framework, combining the strengths of multiple automation approaches to ensure scalability, maintainability, reusability, and robustness. The framework integrates the Page Object Model (POM), Behavior-Driven Development (BDD) using Cucumber, and Data-Driven Testing, supported by Object-Oriented Programming (OOP) principles.

Key Concepts

1. Hybrid Automation Framework : A hybrid framework leverages multiple automation strategies to maximize efficiency and flexibility. In this project, the hybrid approach combines POM, BDD, and data-driven techniques to support scalable and maintainable test development.

Benefits:

- Modular and extensible design.
- High reusability of components.
- Improved readability and collaboration.
- Enhanced test coverage.

2. Page Object Model (POM) : The Page Object Model is a design pattern that encapsulates web elements and their interactions within dedicated page classes. Each page of the Snapdeal application—such as the Home Page, Search Results Page, Product Details Page, and Cart Page—is represented by a separate class.

Benefits:

- Reduces code duplication.
- Simplifies maintenance when UI changes.
- Enhances readability and reusability.

3.Behavior-Driven Development (BDD) with Cucumber : BDD enables the definition of test scenarios in a human-readable format using Gherkin syntax (Given, When, Then, And, But). This approach bridges the gap between technical and non-technical stakeholders.

Benefits:

- Improved communication and collaboration.
- Clear mapping between requirements and tests.
- Reusable step definitions.

4. Data-Driven Testing : Data-driven testing allows the execution of the same test scenario with multiple datasets. In this framework, external data sources such as Excel files are used to parameterize test inputs (e.g., search keywords or menu selections).

Benefits:

- Increased test coverage.
- Separation of test logic from data.
- Easy updates to test inputs without modifying code.

5. Object-Oriented Programming (OOP) Principles :

OOP Concept	Implementation in Framework	Benefit
Encapsulation	Private WebElements in page classes	Protects data and hides implementation
Abstraction	Base classes and utility methods	Simplifies complex operations
Inheritance	Test classes extending BaseTest	Promotes code reuse
Polymorphism	Browser initialization and utility methods	Provides flexibility
Interfaces	Optional browser or utility contracts	Ensures standardization

Tab 4.1.1

6. Synchronization Strategy : Dynamic web elements are handled using Explicit Waits (WebDriverWait), ensuring that elements are interactable before performing actions. This minimizes flaky tests and enhances reliability.

7. Cross-Browser Testing : The framework supports execution on Google Chrome and Microsoft Edge, with browser selection controlled via the config.properties file. This ensures consistent application behavior across different environments.

8. Reporting and Logging :

- Extent Reports provide detailed and visually rich HTML reports.
- Log4j captures execution logs, aiding debugging and traceability.

- Screenshot capture on failure provides visual evidence for analysis.

4.2 Framework Components :

The automation framework is organized into modular components, each responsible for a specific aspect of test execution. This layered structure promotes separation of concerns, reusability, and ease of maintenance.

Component	Example Files	Purpose
Feature Files	*.feature	Define BDD scenarios
Step Definitions	*Steps.java	Implement test logic
Page Objects	HomePage.java, ProductPage.java	Encapsulate UI interactions
Base/Driver	BaseTest.java, DriverFactory.java	Browser setup and teardown
Hooks	Hooks.java	Pre/Post execution actions
Utilities	ConfigReader, ExcelUtils, WaitUtils	Reusable functions
Configuration	config.properties, pom.xml	Environment and dependency management
Reporting	Extent Reports	Execution reporting
Logging	Log4j	Debugging and traceability
Test Data	Excel/JSON files	Parameterized testing

Tab 4.2.1

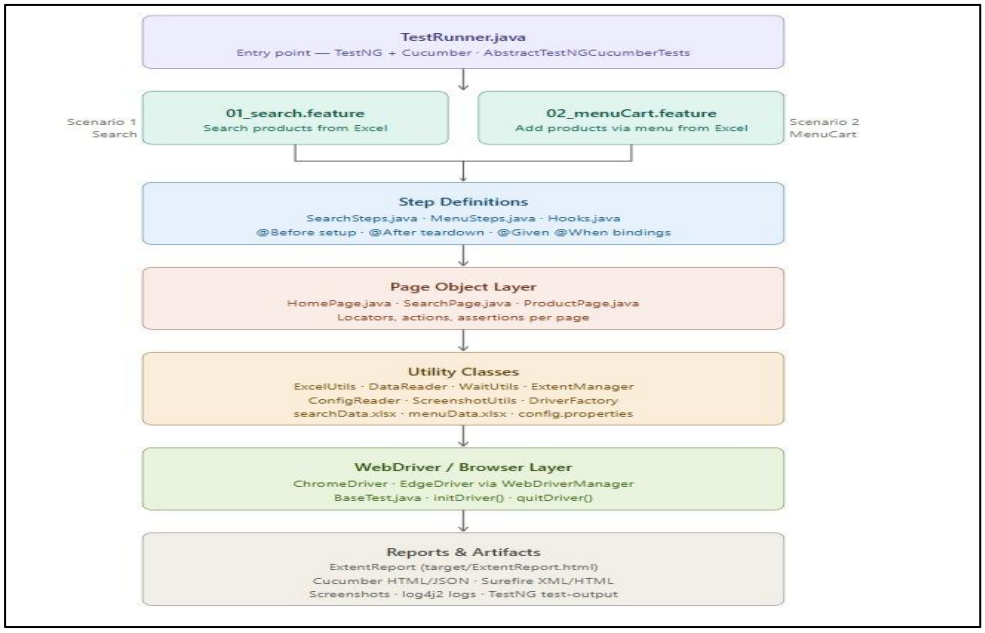


Fig 4.2.1

4.3 Maven Execution & Build Life Cycle :

Apache Maven is used as the build automation and dependency management tool for the Snapdeal automation framework. It provides a standardized project structure, simplifies dependency handling, and supports seamless test execution.

Maven Build Lifecycle Phases :

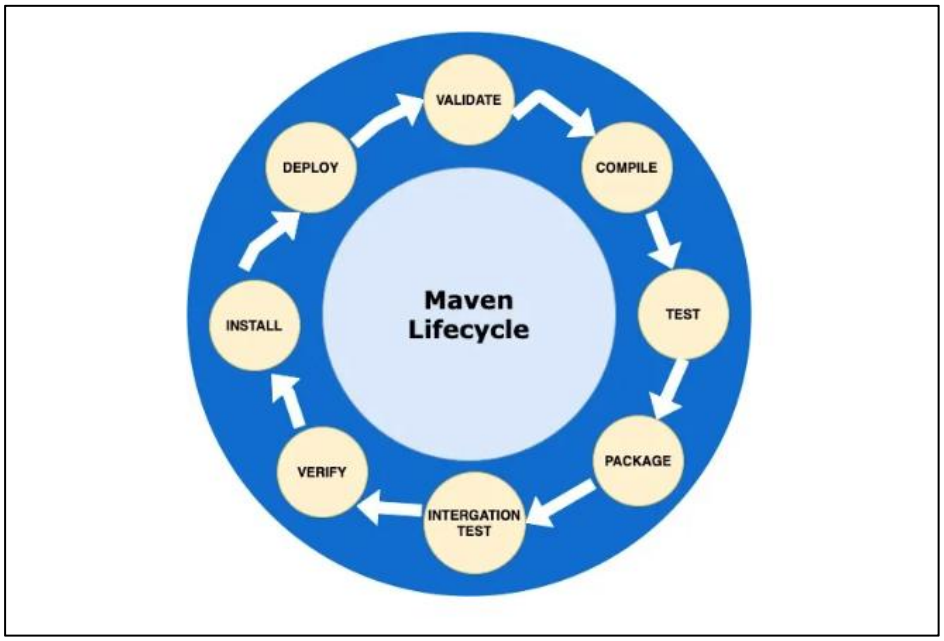


Fig 4.3.1

Phase	Description	Usage in Project
validate	Validates project structure	Ensures correct setup
compile	Compiles source code	Builds framework classes
test	Executes automated tests	Runs Cucumber scenarios
package	Packages compiled code	Generates artifacts
verify	Runs additional checks	Validates test results
install	Installs artifact to local repository	Enables reuse
deploy	Deploys artifact to remote repository	Optional

Tab 4.3.1

Maven Execution Commands :

Command	Purpose
mvn clean	Removes previous build artifacts
mvn compile	Compiles source code
mvn test	Executes automated tests
mvn clean test	Cleans and runs the test suite
mvn verify	Runs tests and validates results

Tab 4.3.2

Section 5 : Testing Strategy

5.1 Test Strategy & Approach :

The test strategy for the Snapdeal automation project is designed to ensure comprehensive validation of critical user journeys on the e-commerce platform. The strategy focuses on delivering high-quality, reliable, and maintainable automated tests using a hybrid automation framework. The approach simulates real-world user interactions, emphasizing functional accuracy, cross-browser compatibility, and data-driven validation.

Objectives of the Test Strategy :

- Validate critical business workflows of the Snapdeal application.
- Ensure end-to-end functional correctness of key user journeys.
- Provide reliable regression testing for future enhancements.
- Achieve maximum test coverage with optimized effort.
- Enable scalable and maintainable automation.
- Generate comprehensive execution reports for analysis.

Test Automation Approach :

1. Behavior-Driven Development (BDD)

- Test scenarios are written in Gherkin syntax using Cucumber.
- Enhances readability and collaboration between stakeholders.

2. Page Object Model (POM)

- Encapsulates web elements and actions within page classes.
- Promotes maintainability and reduces code duplication.

3. Data-Driven Testing

- External Excel files (searchData.xlsx, menuData.xlsx) are used to parameterize test scenarios.
- Enables execution with multiple datasets, increasing coverage.

4. Modular and Reusable Design

- Reusable utilities for driver management, waits, configuration, and reporting.
- Supports easy scalability for additional scenarios.

Test Execution Workflow :

1. TestRunner initiates execution.
2. Feature files define business scenarios.
3. Step definitions map Gherkin steps to automation code.
4. Page objects interact with UI elements.
5. Utilities manage data, waits, and configuration.
6. WebDriver performs browser actions.
7. Reports are generated after execution.

5.2 Test Scenarios Implementation :

The automation framework implements two primary end-to-end scenarios that simulate real user interactions on the Snapdeal platform. These scenarios are aligned with the objectives of the Veeva assignment and validate critical e-commerce functionalities.

Scenario 1 – Product Search

Implemented by: Yadavalli Lakshmi Vyshnavi (Teammate)

Description : This scenario validates the ability of users to search for products and view relevant results.

Key Steps :

1. Launch the Snapdeal homepage.
2. Enter a product keyword in the search bar.
3. Initiate the search.
4. Verify that the search results page is displayed.
5. Validate that the results are relevant to the search keyword.
6. Open a product in a new tab and navigate back.

Validations :

- Presence of search results.
- Relevance of product listings.
- Successful navigation to the product details page.

Scenario 2 – Menu Navigation & Cart Validation

Implemented by: Valluri Gowthami Rushitha (me)

Description : This scenario validates the end-to-end workflow of navigating through product categories, selecting a product, adding it to the cart, and verifying cart details.

Key Steps :

1. Launch the Snapdeal homepage.
2. Hover over a category and select a subcategory.
3. Choose a product from the listing page.
4. Verify navigation to the Product Details Page.
5. Click on the “Add to Cart” button.
6. Validate the cart count.
7. Verify the subtotal and estimated total.
8. Repeat the process using multiple datasets from Excel.

Validations :

- Correct navigation to the product page.
- Successful addition of the product to the cart.
- Accurate cart count and subtotal.
- Data-driven execution across multiple categories.

5.3 Test Coverage & Prioritization Strategy :

The test coverage and prioritization strategy ensures that the most critical functionalities of the Snapdeal application are validated efficiently. The approach focuses on maximizing business value while optimizing test execution time.

Coverage Dimensions :

1. **Functional Coverage :** Validates all major user workflows.
2. **Data Coverage :** Multiple datasets from Excel ensure diverse input combinations.
3. **Browser Coverage :** Execution across Chrome and Edge.
4. **End-to-End Coverage** : Ensures complete business process validation.

Prioritization Strategy : Test scenarios are prioritized based on business impact, risk, and frequency of use.

Priority Levels

Priority	Description	Example
P0 – Critical	Core functionalities that must always work.	Add to Cart, Cart Validation
P1 – High	Frequently used features with significant impact.	Product Search
P2 – Medium	Supporting functionalities.	Navigation and UI checks
P3 – Low	Cosmetic or non-critical features.	Minor UI validations

Tab 5.3.1

Test Coverage Matrix

Feature	Scenario	Priority	Browser Coverage	Data-Driven
Product Search	Scenario 1	P1	Chrome, Edge	Yes
Menu Navigation	Scenario 2	P1	Chrome, Edge	Yes
Add to Cart	Scenario 2	P0	Chrome, Edge	Yes
Cart Validation	Scenario 2	P0	Chrome, Edge	Yes

Tab 5.3.2

Section 6 : Data & Metrics

6.1 Data Driven Testing :

Data-Driven Testing (DDT) is a testing methodology in which the same test scenario is executed multiple times using different sets of input data. In the Snapdeal automation framework, DDT is implemented to enhance test coverage, reusability, and scalability by separating test logic from test data. The framework utilizes external Excel files to parameterize test inputs for both scenarios:

- Scenario 1: Product Search (searchData.xlsx)
- Scenario 2: Menu Navigation & Cart Validation (menuData.xlsx)

This approach allows the automation suite to simulate diverse real-world user inputs without modifying the test scripts.

Objectives of Data-Driven Testing :

- Increase test coverage by executing scenarios with multiple datasets.
- Validate application behavior under valid, edge, and negative conditions.
- Promote reusability of test scripts.
- Simplify maintenance by externalizing test data.
- Enable easy expansion of test scenarios.

Implementation in the Framework :

Component	Description
Excel Files	Store input datasets for different scenarios.
Apache POI	Library used to read data from Excel files.
ExcelUtils Class	Custom utility to fetch data dynamically.
Cucumber Scenario Outline	Enables parameterized execution using examples.
Step Definitions	Iterate through datasets and execute tests.

Tab 6.1.1

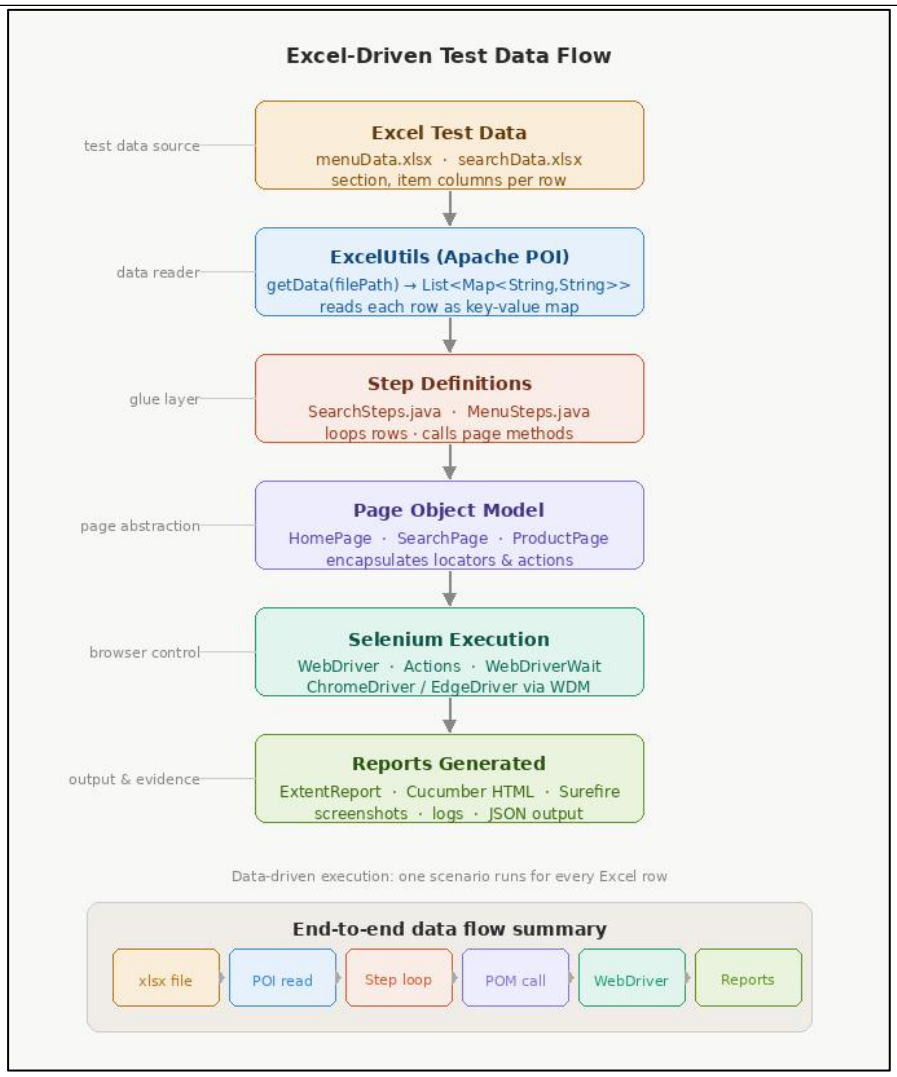


Fig 6.1.1

Excel File Structure :

1. menuData.xlsx – Scenario 2 (Menu Navigation & Cart)

TestCaseID	Section	Item	TestType	ExpectedResult
TC_M01	Kid's Fashion	T-Shirts, Tops & Shirts	Valid	Product added to cart successfully
TC_M02	Home & Kitchen	Kitchen Tools	Valid	Product added to cart successfully
TC_M03	Books	GATE	Edge	Navigation handled correctly
TC_M04	Electronics	Cameras	Edge	No navigation
TC_M05	Sports & Fitness	Very niche item	Negative	No navigation
TC_M06	Toys & Games	Remote Control Toys	Negative	No Mouse Hover

Tab 6.1.2

6.2 Test Data Management Strategy :

A robust Test Data Management (TDM) strategy ensures that test data is accurate, consistent, maintainable, and secure. In this project, test data is managed externally through structured Excel files, enabling efficient organization and reuse.

Key Objectives :

- Maintain data integrity and consistency.
- Enable easy updates and scalability.
- Support multiple test environments.
- Ensure clear classification of test cases.
- Facilitate traceability between data and test scenarios.

Data Organization :

Aspect	Strategy
Data Source	External Excel files (.xlsx).
Data Segregation	Separate files for each scenario.
Data Classification	Valid, Edge, and Negative cases.
Unique Identification	Each dataset assigned a TestCaseID.
Expected Results	Included to validate outcomes.
Version Control	Managed through Git for traceability.

Tab 6.2.1

Data Lifecycle :

Data Creation → Data Validation → Data Storage → Test Execution → Maintenance → Version Control

Data Validation and Maintenance :

- Validation: Data is reviewed before execution to ensure correctness.
- Maintenance: Updates are performed without modifying automation scripts.
- Reusability: The same datasets can be reused across regression cycles.
- Backup & Versioning: Maintained through Git repositories.

Handling Negative Data :

Negative datasets are included to verify the system's robustness. The framework ensures:

- Graceful handling of invalid inputs.
- No unexpected application crashes.
- Proper logging and reporting of failures.

6.3 Test Execution Metrics & Analysis :

Test execution metrics provide quantitative insights into the effectiveness, stability, and quality of the automation framework. These metrics help stakeholders evaluate test performance and support data-driven decision-making.

Key Objectives :

- Measure the effectiveness of the automation suite.
- Monitor test stability and reliability.
- Provide actionable insights for continuous improvement.
- Support quality assurance and reporting.

Key Metrics : (For my work : Scenario 2)

Metric	Description	Output
Total Test Cases	Number of datasets executed.	6
Passed Test Cases	Tests that met expected results.	6
Failed Test Cases	Tests that did not meet expectations.	0
Pass Percentage	(Passed / Total) × 100	100%
Execution Time	Total time taken for execution.	1 min 38 sec
Defect Density	Number of defects per scenario.	0 defects
Browser Coverage	Number of browsers tested.	Chrome, Edge
Data Coverage	Valid, Edge, Negative datasets executed.	100%

Tab 6.3.1

Overall Metrics Table (For 2 Scenarios) :

Scenario	Total	Passed	Failed	Pass %	Execution Time
Scenario 1 – Search	7	7	0	100%	1 min 05 seconds
Scenario 2 – Menu Cart	6	6	0	100%	1 min 38 seconds
Overall	13	13	0	100%	2 min 43 seconds

Tab 6.3.2

Metrics Visualization :

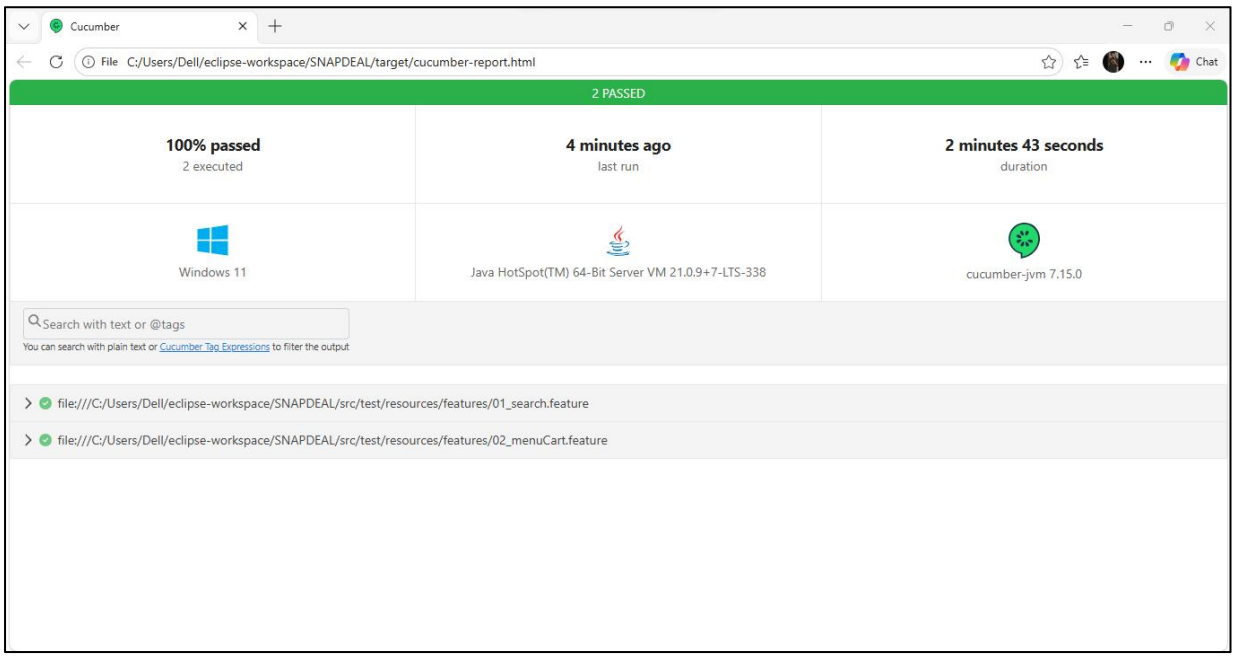


Fig 6.3.1

Analysis of Results :

- High Pass Rate: Indicates stability and reliability of the framework.
- Failures: Typically associated with negative test cases or environmental issues.
- Edge Case Handling: Demonstrates robustness of the application.
- Cross-Browser Consistency: Confirms compatibility across environments.

Section 7 : Advanced Capabilities

7.1 Cross Browser Support :

Cross-browser support ensures that the Snapdeal web application behaves consistently across different web browsers. Since end users may access the application using various browsers, validating functionality across multiple environments is essential for delivering a reliable user experience.

In this project, cross-browser testing is implemented using Selenium WebDriver with support for Google Chrome and Microsoft Edge. The browser selection is dynamically controlled through the config.properties file, enabling flexible and scalable test execution without modifying the test scripts.

Objectives :

- Validate application behavior across different browsers.
- Ensure UI and functional consistency.
- Detect browser-specific issues early.
- Enhance test coverage and reliability.
- Support scalability for future browser additions.

Implementation Approach :

1. **Configuration-Based Browser Selection :** The browser type is externalized in the config.properties file:

```
browser=chrome
url=https://www.snapdeal.com
```

By simply changing the browser value (e.g., chrome or edge), the same test suite can be executed across different browsers.

2. WebDriver Initialization : A centralized Driver Factory/BaseTest class initializes the appropriate WebDriver instance based on the configuration:

Role of WebDriverManager :

- Automatically downloads and manages compatible browser drivers.
- Eliminates manual driver configuration.
- Ensures environment consistency across different machines.

7.2 Wait handling & Synchronization strategy :

Modern web applications like Snapdeal are highly dynamic, with elements loading asynchronously. Without proper synchronization, automation scripts may become flaky and unreliable. To address this, the framework implements a robust wait handling strategy using Selenium's synchronization mechanisms.

Objectives :

- Ensure elements are available before interaction.
- Reduce flaky test failures.
- Improve execution stability.
- Optimize test performance.

Types of Waits Used :

1. Explicit Wait : Explicit waits are the primary synchronization mechanism used in the framework. They wait for specific conditions before proceeding.

Common Conditions Used:

- visibilityOfElementLocated
- elementToBeClickable
- presenceOfElementLocated
- titleContains
- invisibilityOfElementLocated

2. Implicit Wait : A short implicit wait is configured during driver initialization to handle minor delays:

3. Fluent Wait : For elements with unpredictable loading times, Fluent Wait can be used with polling intervals:

Synchronization Techniques Used in Scenario 2 :

Action	Wait Strategy
Menu Hover	Explicit wait for visibility
Subcategory Click	Element to be clickable
Product Selection	Presence of element
Add to Cart	Clickable condition
Cart Validation	Visibility of cart elements
Tab Switching	Wait for number of windows

Tab 7.2.1

7.3 Error handling Strategy :

A robust error handling strategy is essential to ensure that automation tests fail gracefully and provide meaningful insights for debugging. The Snapdeal automation framework incorporates structured exception handling, logging, and screenshot capture to enhance reliability and maintainability.

Objectives :

- Prevent abrupt test termination.
- Provide meaningful diagnostic information.
- Enhance debugging and root cause analysis.
- Ensure graceful handling of unexpected scenarios.

Common Exceptions Handled :

Exception	Cause	Handling Strategy
NoSuchElementException	Element not found	Explicit waits and logging
TimeoutException	Element not loaded in time	Increase wait and capture screenshot
StaleElementReferenceException	DOM refreshed	Re-locate element
ElementClickInterceptedException	Element blocked	Wait until clickable
WebDriverException	Browser issues	Reinitialize driver

Tab 7.3.1

- Screenshot Capture on Failure

- Logging with Log4j
- Integration with Cucumber Hooks
- Assertion Strategy

Error Handling Workflow :

Test Step → Exception Occurs → Logging → Screenshot Capture → Report Update → Graceful Termination

Section 8 : Reporting & Monitoring

8.1 Reporting & Logging :

Reporting and Logging are essential components of an effective automation framework. They provide visibility into test execution, facilitate debugging, and enable stakeholders to understand the overall quality and stability of the application. In the Snapdeal automation project, a robust reporting and logging mechanism has been implemented using Extent Reports, Cucumber Reports, Maven Surefire Reports, and Log4j.

While reporting focuses on presenting execution outcomes in a structured and visual format, logging captures detailed runtime information that aids in diagnosing failures and understanding the execution flow.

Objectives :

- Provide clear and comprehensive execution results.
- Enable quick identification of failures and root causes.
- Maintain traceability between test scenarios and outcomes.
- Support stakeholder communication through visually rich reports.
- Facilitate debugging and maintenance via detailed logs.

Reporting Tools Used :

Tool	Purpose	Output Format
Extent Reports	Detailed and visually rich execution reports with screenshots.	HTML
Cucumber Reports	Scenario-wise BDD execution results.	HTML/JSON
Maven Surefire Reports	TestNG execution summaries.	HTML/XML
Screenshots	Visual evidence of failures.	PNG

Tab 8.1.1

1. Extent Reports : Extent Reports provide interactive and user-friendly HTML reports that include step-level execution details, screenshots, timestamps, and overall execution summaries.

Key Features :

- Step-by-step execution status (Pass/Fail/Skip).
- Screenshot attachment for failed steps.
- Test categorization by scenario.
- Execution timestamps and duration.
- Graphical representation of results.
- System and environment information.

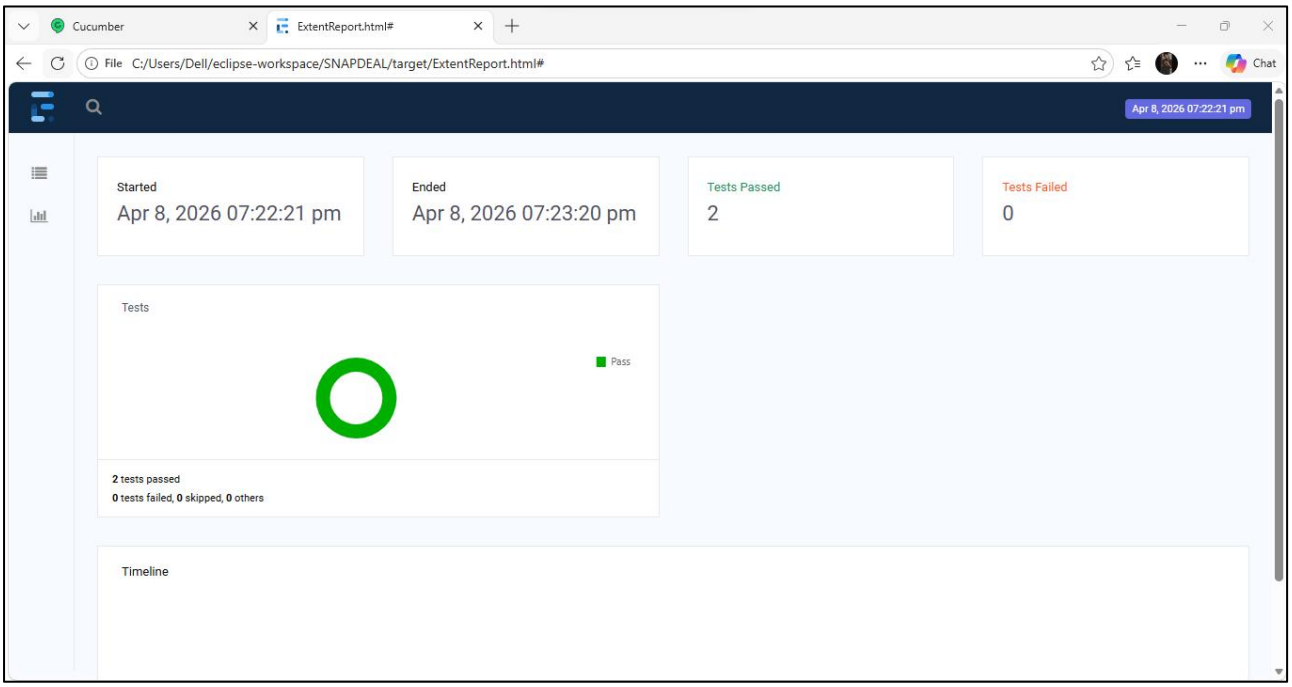


Fig 8.1.1

2. Cucumber Reports : Cucumber Reports provide a behavior-driven view of test execution, mapping directly to feature files written in Gherkin syntax.

Key Features :

- Scenario and step-level results.
- Mapping of Given-When-Then steps.
- Pass/Fail statistics.
- Execution duration.
- JSON and HTML output for further analysis.

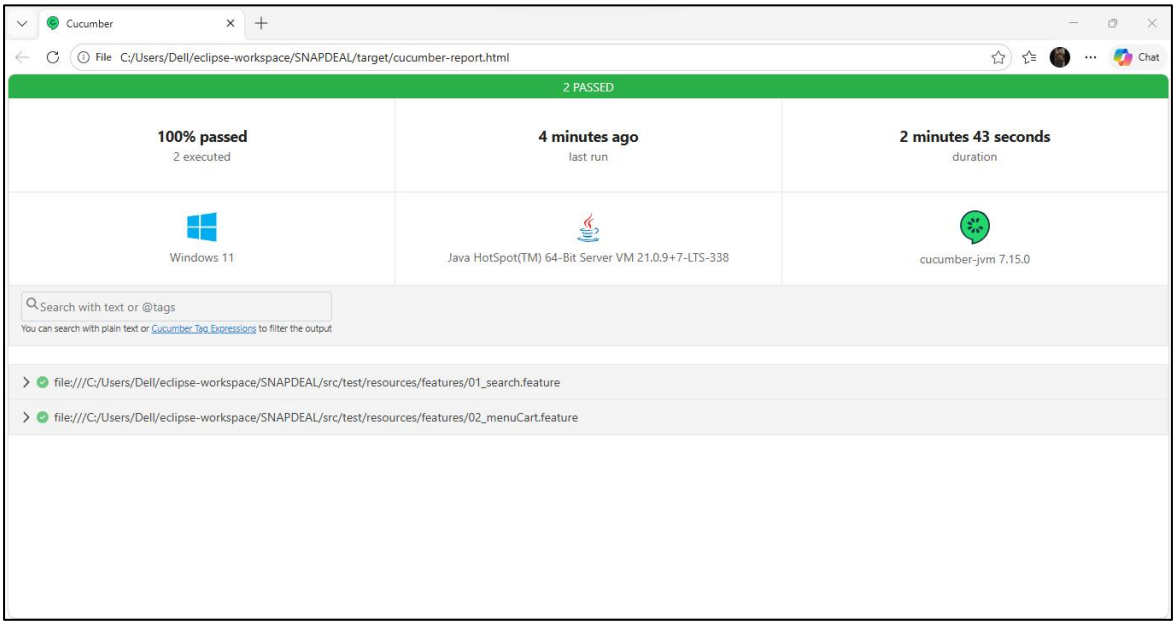


Fig 8.1.2

3. Maven Surefire Reports : The Maven Surefire Plugin generates execution reports when tests are run using Maven. These reports provide a summary of test results, including the number of tests executed, passed, failed, and skipped.

Key Features :

- TestNG-based execution summaries.
- XML and HTML report generation.
- Integration with CI/CD tools.
- Historical result tracking.

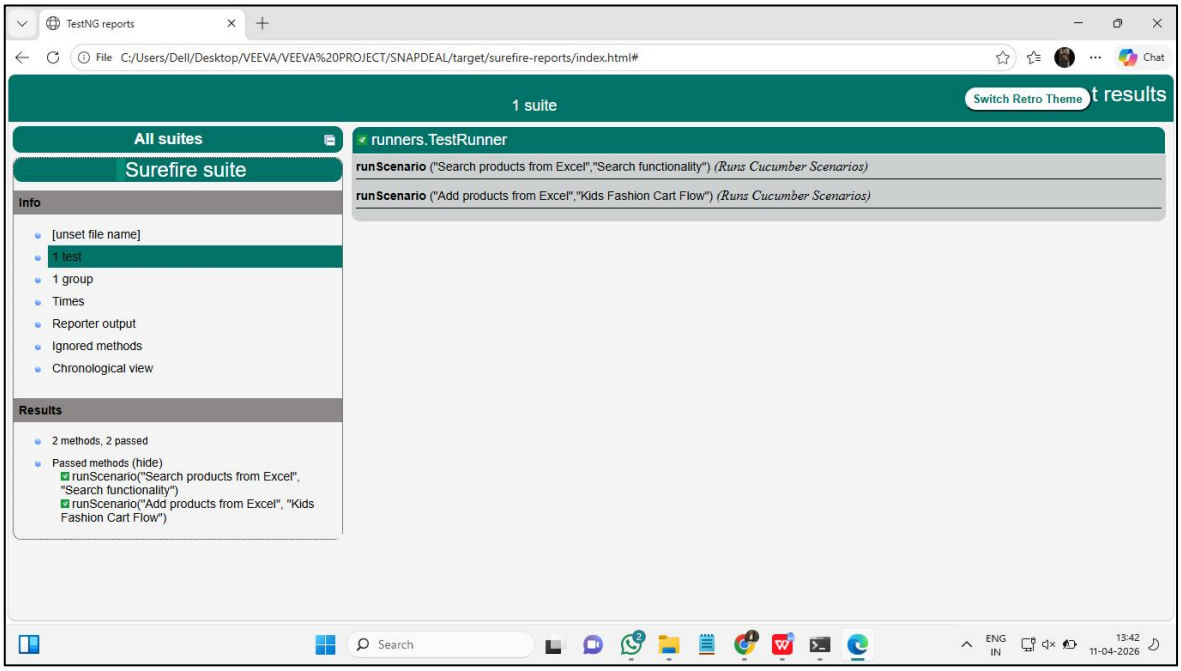
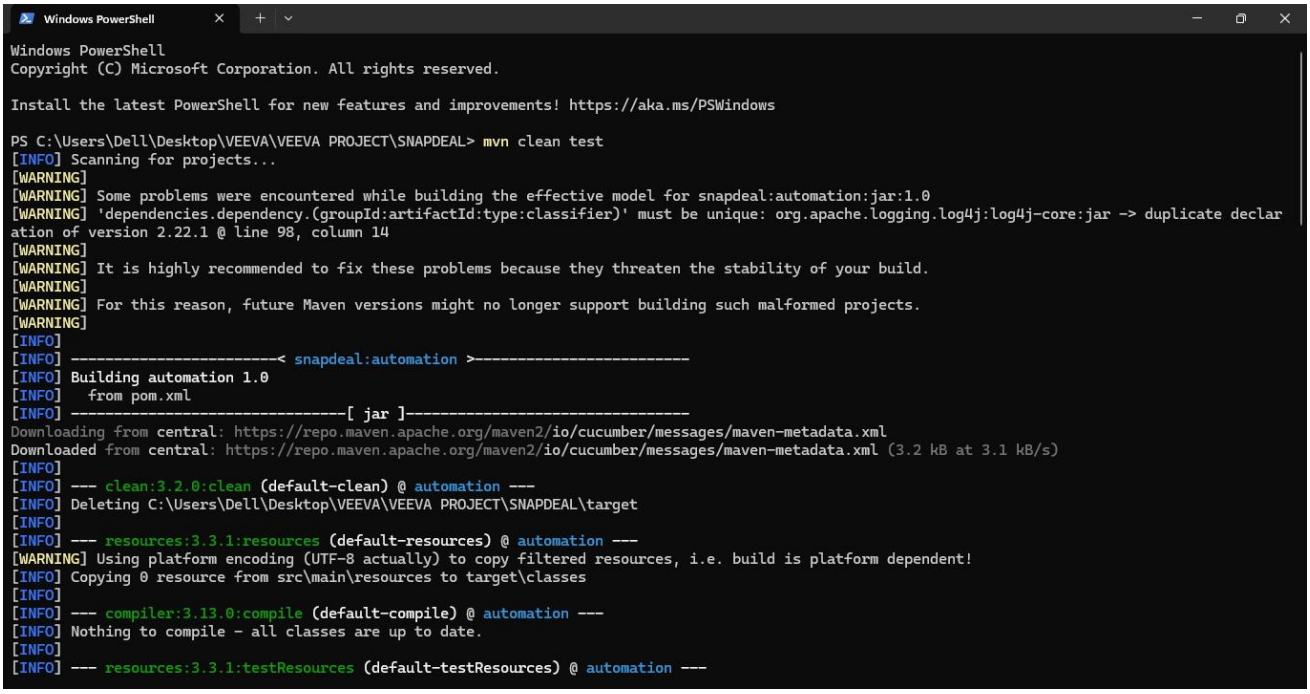


Fig 8.1.3

4. Logging with Log4j : Log4j is used to capture detailed runtime information during test execution. Logging helps in diagnosing issues, understanding execution flow, and maintaining traceability.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Dell\Desktop\VEEVA\VEEVA PROJECT\SNAPDEAL> mvn clean test
[INFO] Scanning for projects...
[WARNING]
[WARNING] Some problems were encountered while building the effective model for snapdeal:automation:jar:1.0
[WARNING] 'dependencies.dependency.(groupId:artifactId:type:classifier)' must be unique: org.apache.logging.log4j:log4j-core:jar -> duplicate declar
ation of version 2.22.1 @ line 98, column 14
[WARNING]
[WARNING] It is highly recommended to fix these problems because they threaten the stability of your build.
[WARNING]
[WARNING] For this reason, future Maven versions might no longer support building such malformed projects.
[WARNING]
[INFO]
[INFO] ------------------------< snapdeal:automation >-------------------------
[INFO] Building automation 1.0
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
Downloading from central: https://repo.maven.apache.org/maven2/io/cucumber/messages/maven-metadata.xml
Downloaded from central: https://repo.maven.apache.org/maven2/io/cucumber/messages/maven-metadata.xml (3.2 kB at 3.1 kB/s)
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ automation ---
[INFO] Deleting C:\Users\Dell\Desktop\VEEVA\VEEVA PROJECT\SNAPDEAL\target
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ automation ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ automation ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ automation ---
```

Fig 8.1.4

Logging Levels :

Level	Purpose	Example
INFO	General execution steps	"Navigating to Snapdeal homepage."
DEBUG	Detailed technical information	"Locating Add to Cart button."
WARN	Potential issues	"Element found but not clickable immediately."
ERROR	Failures and exceptions	"Add to Cart button not found."
FATAL	Critical system failures	"WebDriver initialization failed."

Tab 8.1.2

5.Screenshot Capture : Screenshots are automatically captured for failed scenarios and attached to reports, providing visual evidence for debugging.

Reporting & Logging Workflow :

Test Execution → Logging (Log4j) → Screenshot Capture → Report Generation (Extent, Cucumber, Surefire) → Stakeholder Analysis

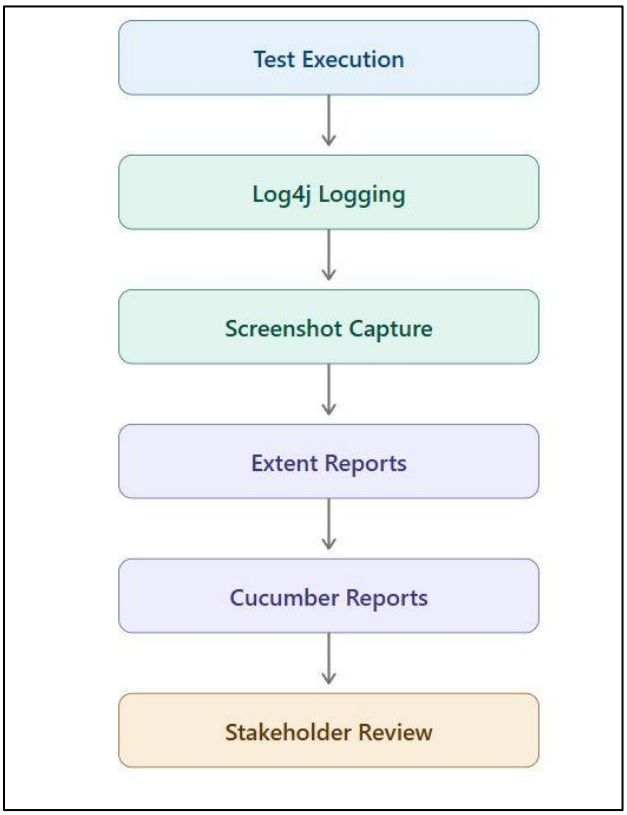


Fig 8.1.5

8.2 Execution Reports & Insights :

Execution Reports & Insights provide quantitative and qualitative information about test execution. These insights help stakeholders assess application quality, identify trends, and make informed decisions regarding releases.

Scenario-Wise Execution Summary :

Scenario	Owner	Total Iterations	Passed	Failed	Pass %
Scenario 1 – Product Search	Vyshnavi	7	7	0	100%
Scenario 2 – Menu Navigation & Cart	Rushitha	6	6	0	100%
Overall	-	13	13	0	100%

Tab 8.2.1

Insights Derived from Execution :

1. Functional Stability : High pass rates indicate stable application behavior & Successful execution of Scenario 2 demonstrates robustness of the cart functionality.

2. Effectiveness of Data-Driven Testing : Execution across valid, edge, and negative datasets ensures comprehensive validation & Confirms the framework's scalability and flexibility.

3. Cross-Browser Consistency : Consistent results across Chrome and Edge validate compatibility & No major browser-specific defects were observed.

4. Defect Identification : Failures in negative scenarios highlight the system's handling of invalid inputs & Logs and screenshots facilitate quick root cause analysis.

5. Performance Observations : Execution time remains within acceptable limits & Efficient synchronization strategies minimize delays.

- Execution reports provide traceability and ensures transparency and facilitates effective communication with stakeholders, including evaluators and project reviewers.

Section 9 : Configuration & Maintainability

9.1 Configuration Management :

Configuration Management is a critical aspect of a robust automation framework, enabling flexible and environment-independent test execution. It involves externalizing environment-specific parameters and managing them systematically to ensure consistency, maintainability, and ease of execution across different setups.

In the Snapdeal automation project, configuration management is implemented using a config.properties file, centralized configuration reader utilities, and Maven profiles/system properties. This approach eliminates hardcoded values and allows seamless switching between browsers and environments.

Objectives :

- Externalize environment-specific parameters.
- Enable flexible and dynamic test execution.
- Support cross-browser and multi-environment testing.
- Simplify maintenance and updates.
- Ensure consistency and reproducibility across executions.
- Facilitate integration with CI/CD pipelines.

Key Configuration Elements :

Configuration Item	Description
Application URL	URL of the Snapdeal website.
Browser Type	Specifies the browser for execution.
Timeout Values	Defines wait durations for synchronization.
Report Paths	Location for storing execution reports.
Screenshot Paths	Directory for failure screenshots.
Environment	Supports different execution environments.

Tab 9.1.1

Configuration Reader Utility : A centralized ConfigReader class is used to read configuration values, ensuring easy access throughout the framework.

Integration with CI/CD (GitHub Actions) : Configuration values can be dynamically passed through CI pipelines:

Configuration Management Workflow :

config.properties → ConfigReader → DriverFactory/BaseTest → Test Execution → Reports

9.2 Framework Scalability & Reusability :

Scalability and Reusability are essential qualities of a well-designed automation framework. Scalability ensures that the framework can accommodate new features, scenarios, and environments with minimal effort, while reusability allows existing components to be leveraged across multiple test cases, reducing redundancy and maintenance overhead.

The Snapdeal automation framework achieves these qualities through a modular architecture, Page Object Model (POM), data-driven testing, and centralized utilities.

Objectives :

- Enable easy addition of new test scenarios.
- Promote reuse of existing components.
- Reduce maintenance effort.
- Support execution across multiple environments and browsers.
- Facilitate integration with CI/CD pipelines.

Key Design Principles Supporting Scalability :

1. Modular Architecture : The framework is organized into distinct layers such as feature files, step definitions, page objects, utilities, and configuration files. This separation of concerns allows new components to be added without impacting existing functionality.

2. Page Object Model (POM) : Each web page is represented by a dedicated class encapsulating its elements and actions. When UI changes occur, only the corresponding page class needs to be updated.

3. Reusable Utility Classes :

Utility	Purpose
BaseTest	WebDriver initialization and teardown.
ConfigReader	Reads configuration properties.
ExcelUtils	Handles data-driven testing.
WaitUtils	Manages synchronization.
ScreenshotUtils	Captures screenshots on failure.
ExtentManager	Manages report generation.

Tab 9.2.1

These utilities can be reused across all scenarios, minimizing code duplication

Cross-Browser and Environment Scalability : The framework supports scalability across:

- Browsers: Chrome and Edge (extendable to Firefox).
- Environments: QA, Staging, and Production.
- Data Sets: Multiple valid, edge, and negative test cases.

Section 10 : CI/CD Integration

10.1 Github Actions :

Continuous Integration and Continuous Deployment (CI/CD) are essential practices in modern software development that enable automated building, testing, and validation of applications. In the Snapdeal automation project, GitHub Actions is utilized to implement Continuous Integration (CI), ensuring that automated tests are executed consistently whenever changes are made to the codebase.

Although Jenkins was not implemented, GitHub Actions provides a robust and industry-recognized alternative for automating test execution, making the framework CI-ready and aligned with real-world DevOps practices.

Objectives of CI/CD Integration :

- Automate execution of the automation test suite.

- Ensure early detection of defects.
- Maintain code quality and stability.
- Enable consistent and repeatable builds.
- Provide rapid feedback to developers and stakeholders.
- Support collaboration through version control.
- Demonstrate DevOps readiness for enterprise environments.

GitHub Actions :

GitHub Actions is a cloud-based CI/CD platform integrated with GitHub repositories that allows automation of workflows such as building, testing, and deploying applications. It uses YAML-based workflow files to define when and how tasks should be executed.

CI Workflow in the Snapdeal Automation Framework :

The CI pipeline automates the following steps:

Code Commit → GitHub Repository → GitHub Actions Triggered →
Environment Setup → Maven Build → Test Execution →
Report Generation → Artifact Upload → Execution Status

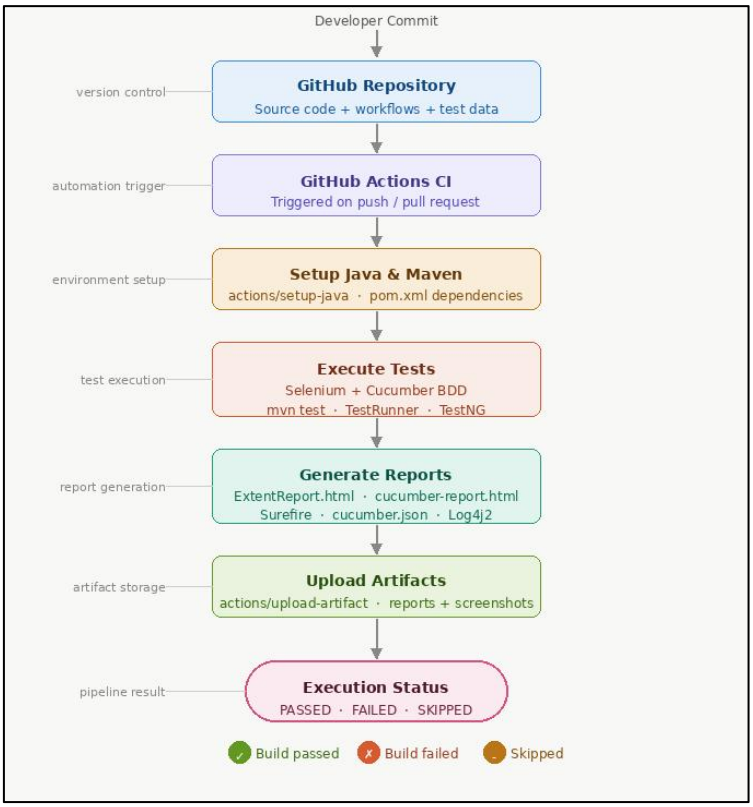


Fig 10.1.1

Explanation of Workflow Steps :

Step	Description
Checkout Repository	Retrieves the latest code from GitHub.
Set Up Java	Configures the Java Development Kit required for the project.
Cache Maven Dependencies	Speeds up builds by caching dependencies.
Install Browsers	Ensures the required browsers are available for testing.
Run Automation Tests	Executes the test suite using Maven.
Upload Artifacts	Stores Extent and Cucumber reports for review.

Tab 10.1.1

Artifact Management :

Generated reports are uploaded as artifacts, enabling stakeholders to download and review execution results directly from GitHub.

Artifact	Description
Extent Report	Detailed HTML report with screenshots.
Cucumber Report	BDD execution summary.
Surefire Reports	Maven test execution results.
Logs & Screenshots	Evidence for debugging failures.

Tab 10.1.2

Section 11 : Real World Considerations

11.1 Risk Analysis :

Risk analysis is a proactive approach to identifying potential issues that may impact the success of automation testing. By evaluating these risks early, appropriate mitigation strategies can be implemented to ensure reliable and stable test execution.

Key Risks and Mitigation Strategies :

Risk Category	Description	Impact	Mitigation Strategy
Dynamic UI Changes	Frequent updates to Snapdeal's UI may break locators.	High	Use robust locators and POM design.
Element Synchronization Issues	Delayed loading of elements can cause flaky tests.	High	Implement explicit and fluent waits.
Test Data Inconsistency	Invalid or outdated data may lead to incorrect results.	Medium	Centralized and validated Excel datasets.
Browser Compatibility Issues	Differences in browser behavior may affect execution.	Medium	Cross-browser testing using WebDriverManager.
Network or Environment Failures	Internet or server issues may interrupt execution.	Medium	Retry mechanisms and stable test environments.
Third-Party Dependency Changes	Changes in external libraries or drivers.	Low	Use Maven dependency management and version control.
False Positives/Negatives	Incorrect test outcomes due to unstable scripts.	Medium	Robust assertions and synchronization strategies.
CI Pipeline Failures	Build or execution failures in GitHub Actions.	Medium	Proper workflow configuration and logging.

Tab 11.1.1

Risk Severity Matrix :

Severity	Description	Example
High	Can cause major test failures.	UI changes, synchronization issues
Medium	May affect execution but manageable.	Data inconsistencies, browser issues
Low	Minimal impact on overall execution.	Dependency updates

Tab 11.1.2

11.2 Challenges faced & Solutions :

During the development of the Snapdeal automation framework, several practical challenges were encountered. Addressing these challenges significantly enhanced the robustness, reliability, and scalability of the framework. The key challenges and their corresponding solutions are outlined below:

Key Challenges and Implemented Solutions :

1. Dynamic Web Elements

Challenge: Many web elements on the Snapdeal website were dynamic in nature, with attributes such as IDs and classes changing frequently or loading asynchronously. This led to difficulties in reliably locating elements during test execution.

Solution Implemented:

- Utilized stable XPath and CSS locators based on unique and consistent attributes.
- Implemented explicit waits using WebDriverWait and ExpectedConditions to ensure elements were present and interactable before performing actions.
- Adopted the Page Object Model (POM) to centralize locator management, simplifying maintenance when UI changes occur.

2. Handling Hover-Based Menus

Challenge: Snapdeal's navigation menu requires mouse hover actions to reveal subcategories, which cannot be handled using standard click operations.

Solution Implemented:

- Employed Selenium's Actions class to simulate mouse hover events.
- Ensured element visibility before interaction using explicit waits.
- Encapsulated hover functionality within reusable methods in the HomePage class.

3. Multiple Browser Tabs

Challenge: Selecting a product often opened the Product Details Page in a new browser tab, causing the WebDriver to lose context of the active window.

Solution Implemented:

- Managed window handles to switch control from the parent window to the newly opened tab.
- Ensured seamless navigation back to the original window after validation.

4. Synchronization Issues

Challenge: Timing mismatches between the automation scripts and the application's dynamic content resulted in flaky tests and intermittent failures.

Solution Implemented:

- Developed a reusable WaitUtils class to centralize synchronization logic.
- Used explicit waits such as visibilityOfElementLocated and elementToBeClickable.
- Applied minimal implicit waits for handling minor delays.

6. Cart Total Validation

Challenge: Validating the cumulative cart total when multiple products were added required accurate calculation and verification logic.

Solution Implemented:

- Implemented custom logic to capture individual product prices and compute the expected subtotal.
- Compared the calculated value with the displayed estimated total to ensure accuracy.
- Added assertions to validate cart count and pricing consistency.

7. Cross-Browser Execution

Challenge: Ensuring consistent behavior of the automation framework across different browsers was essential for real-world applicability.

Solution Implemented:

- Centralized WebDriver initialization using a DriverFactory/BaseTest class.
- Utilized WebDriverManager to automatically manage browser drivers.
- Enabled browser selection through the config.properties file and Maven parameters.

8. Failure Diagnosis

Challenge: Identifying the root cause of test failures was difficult without sufficient execution details and evidence.

Solution Implemented:

- Integrated Log4j for detailed logging at various levels.
- Implemented automatic screenshot capture for failed scenarios.
- Attached screenshots and logs to Extent Reports and Cucumber Reports for enhanced traceability.

11.3 Limitations & Future Enhancements :

While the Snapdeal automation framework demonstrates strong alignment with industry standards, certain limitations exist due to project scope and environmental constraints. Identifying these limitations provides opportunities for future enhancements and continuous improvement.

Limitations

- Limited browser support, currently restricted to Google Chrome and Microsoft Edge.
- Absence of parallel test execution, resulting in longer execution times.
- Framework focuses primarily on UI-based functional testing, with no API testing integration.
- Execution depends on the live Snapdeal website, making tests susceptible to network issues and external changes.
- Limited performance and load testing capabilities.
- Absence of security testing to identify vulnerabilities.
- Limited environment support, with no dedicated staging or production configurations.
- Lack of containerization (e.g., Docker) for consistent execution environments.
- Reporting is limited to Extent and Cucumber reports, without advanced analytics dashboards.

Future Enhancements

- Extend browser coverage to include Mozilla Firefox and Safari.
- Implement parallel test execution using Selenium Grid or TestNG parallelization to reduce execution time.
- Integrate API testing using tools such as Rest Assured for end-to-end validation.
- Incorporate performance testing using tools like Apache JMeter.
- Implement security testing practices.

- Support multiple environments (QA, Staging, Production) through enhanced configuration management.
- Containerize the framework using Docker for consistent and portable execution.
- Enhance reporting by integrating Allure Reports or custom analytics dashboards.
- Enable scheduled executions and advanced pipeline stages within GitHub Actions.
- Add visual regression testing to detect UI inconsistencies.
- Expand the framework to support mobile automation using Appium

Section 12 : Business Justification

12.1 Design Decisions (Why Snapdeal over Walmart) :

The selection of the Application Under Test (AUT) plays a crucial role in the successful design and implementation of an automation framework. While both Snapdeal and Walmart are prominent e-commerce platforms offering rich functionalities, a careful evaluation of Walmart's limitations revealed that Snapdeal provides a more suitable and automation-friendly environment for demonstrating the objectives of this assignment.

By analyzing the constraints associated with Walmart, it becomes evident that these limitations translate into significant advantages when using Snapdeal for automation testing.

Walmart Limitations and Snapdeal Advantages :

1. Advanced Bot Detection and Anti-Automation Mechanisms

Walmart Limitation: Walmart employs stringent bot detection mechanisms, including:

- CAPTCHA challenges.
- Rate limiting.
- Automated traffic detection.
- Session validation and blocking.

These mechanisms can interrupt Selenium-based automation and lead to frequent test failures.

Snapdeal Advantage:

- Snapdeal has comparatively fewer automation restrictions, allowing smoother interaction with Selenium WebDriver.
- Enables uninterrupted end-to-end test execution, ensuring reliability and consistency of results.

2. Frequent UI Changes and Dynamic Rendering

Walmart Limitation:

- Walmart's website undergoes frequent UI updates and uses highly dynamic components.
- This results in unstable locators and increased maintenance effort for automation scripts.

Snapdeal Advantage:

- Snapdeal maintains a relatively stable and consistent UI structure, which simplifies locator strategies.
- Reduces script maintenance and enhances framework reliability.

3. Complex Navigation and Localization

Walmart Limitation:

- Walmart's navigation and content are heavily influenced by geolocation and localization, leading to variations in categories, product listings, and layout.
- Such inconsistencies complicate the creation of reusable and deterministic test scenarios.

Snapdeal Advantage:

- Snapdeal offers consistent navigation and category structures, enabling the creation of reliable and reusable automation scripts.
- Ideal for demonstrating data-driven testing across multiple categories.

4. Ethical and Compliance Considerations

Walmart Limitation:

- Due to strict terms of service and anti-bot policies, automating Walmart may raise ethical and compliance concerns.
- Excessive automation attempts may lead to IP blocking or account restrictions.

Snapdeal Advantage:

- Snapdeal provides a more automation-friendly environment suitable for academic and demonstration purposes.
- Enables responsible automation within acceptable usage limits.

5. Increased Automation Complexity

Walmart Limitation:

- The presence of advanced security layers and complex workflows increases the complexity of automation, diverting focus from demonstrating framework design and engineering concepts.

Snapdeal Advantage:

- Snapdeal provides an optimal balance of functional richness and automation simplicity, enabling effective demonstration of:
    - Hybrid framework architecture.
    - Page Object Model (POM).
    - Behavior-Driven Development (BDD).
    - Data-driven testing.
    - Cross-browser execution.
    - CI integration using GitHub Actions.

Section 13 : Outcomes

13.1 Learning Outcomes :

The Snapdeal automation project provided valuable hands-on experience in designing and implementing a robust automation framework. The key learning outcomes include:

- **Understanding of Automation Framework Design:** Gained practical knowledge of building a hybrid automation framework using Selenium, Cucumber (BDD), TestNG, and the Page Object Model (POM).

- **Application of OOP Concepts:** Implemented core object-oriented principles such as encapsulation, abstraction, inheritance, and polymorphism to enhance maintainability and scalability.
- **Behavior-Driven Development (BDD):** Learned to write business-readable test scenarios using Gherkin syntax and map them to executable step definitions.
- **Data-Driven Testing:** Acquired experience in handling external test data using Excel and Apache POI, including valid, edge, and negative test cases.
- **Cross-Browser Testing:** Executed tests across Chrome and Microsoft Edge, ensuring consistent application behavior.
- **Synchronization and Error Handling:** Implemented explicit waits, logging, and screenshot capture to improve test stability and debugging.
- **Reporting and Monitoring:** Generated comprehensive Extent and Cucumber reports to analyze execution results.
- **CI Integration:** Gained exposure to GitHub Actions for automating test execution and establishing a CI pipeline.
- **Team Collaboration:** Developed teamwork and communication skills by contributing to scenario-based implementation.
- **Real-World Testing Experience**: Simulated real e-commerce user workflows, enhancing practical understanding of software testing in enterprise environments.

13.2 Execution Proof :

To validate the successful execution of the automation framework, the following artifacts serve as execution proof:

- **Extent Report:** Provides a detailed HTML report with step-by-step execution status, screenshots, and summary statistics.
- **Cucumber Report:** Displays scenario-wise execution results aligned with BDD feature files.
- **Maven Surefire Reports:** Offer execution summaries in XML/HTML format, confirming successful test runs.
- **Screenshots:** Automatically captured for failed scenarios, providing visual evidence for debugging.
- **Console Logs:** Execution logs generated using Log4j, detailing each step of the test process.
- **GitHub Actions Workflow:** Demonstrates automated CI execution with downloadable report artifacts.

- **Pass/Fail Summary:** Scenario-wise results confirming the successful validation of both Scenario 1 and Scenario 2.

The successful implementation and execution of the Snapdeal automation framework are demonstrated through comprehensive evidence, including execution reports, logs, and recorded videos. The following Google Drive links provide visual confirmation of the automated scenarios and CI pipeline execution:

- Execution through runner class:

  https://drive.google.com/file/d/1mLTcZIwqI_Rl8PUfYQk58O4ii34kj0Mo/view?usp=sharing

- Execution through POM.xml :

  https://drive.google.com/file/d/10fioSjsTYeFlwJG8r2umcLcNtZ8XI10L/view?usp=sharing

- Execution through CMD :

  https://drive.google.com/file/d/1GihC57oYxoDjAriuITEz0WVCcmmBuDoF/view?usp=sharing

These artifacts provide verifiable evidence of the framework's functionality, robustness, and alignment with industry best practices.

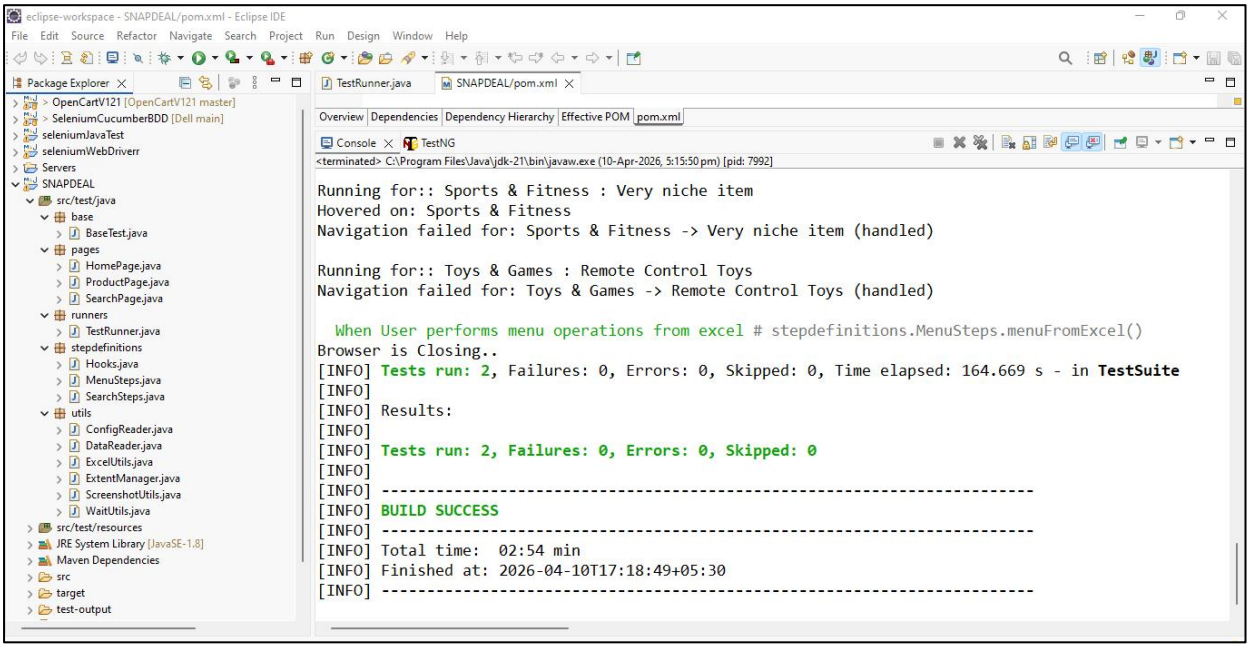


Fig 13.2.1

13.3 Conclusion :

The Snapdeal automation project successfully demonstrates the design and implementation of a robust, scalable, and maintainable hybrid automation framework. By integrating Selenium WebDriver, Cucumber BDD, TestNG, and data-driven testing, the framework effectively validates critical e-commerce workflows such as product search and menu-based cart operations.

Key achievements of the project include:

- Development of a modular and reusable automation framework.
- Successful implementation of data-driven and cross-browser testing.
- Integration of comprehensive reporting and logging mechanisms.
- Establishment of a CI pipeline using GitHub Actions.
- Clear differentiation of team contributions, with Scenario 2 highlighting advanced automation capabilities.

Overall, the project reflects strong adherence to industry best practices and demonstrates readiness for enterprise-level automation environments, aligning closely with the expectations of Veeva Systems.